

Notación O grande (Big-O)

La **Notación O grande** (Big-O) es una forma de expresar la **complejidad de un algoritmo** en términos del tamaño de la entrada. En otras palabras, describe el **comportamiento del tiempo de ejecución** (o el uso de recursos, como la memoria) de un algoritmo cuando el tamaño de los datos de entrada crece.

La notación O grande no mide el tiempo exacto de ejecución, sino que proporciona una **estimación** de cómo el tiempo de ejecución o el uso de recursos crece a medida que el tamaño de la entrada aumenta. Se enfoca en el **peor caso** o el **escenario más grande**, lo que es útil para comparar la eficiencia de diferentes algoritmos de forma independiente al hardware o al software utilizado.

¿Para qué se usa la Notación O grande?

1. **Comparar la eficiencia de los algoritmos:** Ayuda a evaluar cuál algoritmo es más eficiente, especialmente cuando se manejan grandes volúmenes de datos. Permite saber qué tan rápido o lento crecerá el tiempo de ejecución a medida que el tamaño de los datos aumenta.
2. **Entender la escalabilidad:** Al analizar el rendimiento del algoritmo con grandes conjuntos de datos, puedes prever si será eficiente o si llevará mucho tiempo procesar datos más grandes.
3. **Predecir el peor caso:** La notación O grande generalmente se usa para describir el rendimiento del **peor escenario posible** (es decir, cuando el algoritmo encuentra la entrada más difícil).

Principales Categorías de Complejidad en Notación O grande

1. **O(1) - Constante:** El tiempo de ejecución es independiente del tamaño de la entrada. No importa si tienes 10 elementos o 1 millón, el tiempo de ejecución sigue siendo el mismo.

- Ejemplo: Acceso a un elemento en un array por su índice.

```
int[] arr = {10, 20, 30, 40, 50};
int valor = arr[2]; // Siempre tarda el mismo tiempo, sin importar
                    // el tamaño del arreglo.
```

2. **O(log n) - Logarítmica:** El tiempo de ejecución aumenta lentamente a medida que crece el tamaño de la entrada. Este comportamiento se observa en algoritmos que dividen el problema a la mitad en cada paso.

- Ejemplo: Búsqueda binaria.

```
int busquedaBinaria(int[] arr, int valorBuscado) {
    int inicio = 0, fin = arr.length - 1;
    while (inicio <= fin) {
        int medio = inicio + (fin - inicio) / 2;
        if (arr[medio] == valorBuscado) return medio;
        if (arr[medio] < valorBuscado) inicio = medio + 1;
        else fin = medio - 1;
    }
    return -1;
}
```

3. **O(n) - Lineal:** El tiempo de ejecución crece en proporción directa al tamaño de la entrada. Si duplicas la entrada, el tiempo de ejecución también se duplica.

- Ejemplo: Recorrer un array.

```
void recorrerArray(int[] arr) {
    for (int i = 0; i < arr.length; i++) {
        System.out.println(arr[i]);
    }
}
```

4. **$O(n \log n)$ - Log lineal:** Es una combinación de **$O(n)$** y **$O(\log n)$** . Este tipo de complejidad es común en algoritmos eficientes de ordenamiento como el **Merge Sort** o **Quick Sort**.

- Ejemplo: Ordenamiento con QuickSort o MergeSort.

```
void quickSort(int[] arr, int inicio, int fin) {
    if (inicio < fin) {
        int pivote = particion(arr, inicio, fin);
        quickSort(arr, inicio, pivote - 1);
        quickSort(arr, pivote + 1, fin);
    }
}
```

5. **$O(n^2)$ - Cuadrática:** El tiempo de ejecución aumenta con el cuadrado del tamaño de la entrada. Esto ocurre generalmente en algoritmos ineficientes que usan bucles anidados, como algunos algoritmos de ordenamiento simples (por ejemplo, Bubble Sort).

- Ejemplo: Algoritmo de ordenamiento **Bubble Sort**.

```
void bubbleSort(int[] arr) {
    int n = arr.length;
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

6. **$O(2^n)$ - Exponencial:** El tiempo de ejecución se duplica con cada adición a la entrada. Este tipo de complejidad es muy ineficiente y suele encontrarse en algoritmos recursivos que generan muchas combinaciones, como la resolución de problemas por fuerza bruta.

- Ejemplo: Resolver el problema de la **torre de Hanoi** o algunos problemas de **fuerza bruta**.

```
void torresDeHanoi(int n, char origen, char destino, char auxiliar) {
    if (n == 1) {
        System.out.println("Mover disco 1 desde " + origen + " hasta "
+ destino);
        return;
    }
    torresDeHanoi(n - 1, origen, auxiliar, destino);
    System.out.println("Mover disco " + n + " desde " + origen + "
hasta " + destino);
    torresDeHanoi(n - 1, auxiliar, destino, origen);
}
```

7. **$O(n!)$ - Factorial:** El tiempo de ejecución crece factorialmente con el tamaño de la entrada. Esto ocurre en problemas donde hay que generar y revisar todas las permutaciones o combinaciones posibles, como el problema del agente viajero.

- Ejemplo: Resolver el **problema del viajante**.

¿Cuándo se usa la Notación O grande?

La notación O grande se utiliza principalmente en los siguientes contextos:

1. **Análisis de Algoritmos:** Para evaluar y comparar la eficiencia de diferentes algoritmos en términos de tiempo de ejecución y uso de memoria.
2. **Diseño de Software:** Al diseñar software, es importante elegir algoritmos eficientes, especialmente cuando se espera trabajar con grandes volúmenes de datos.
3. **Optimización de Rendimiento:** Permite a los programadores identificar cuellos de botella y áreas de mejora en un programa al observar cómo escala con el tamaño de los datos de entrada.

Conclusión

La **Notación O grande** es una herramienta fundamental para comprender la **eficiencia** y **escalabilidad** de los algoritmos. Permite predecir cómo se comportará un algoritmo a medida que crece el tamaño de los datos de entrada y es clave para elegir el mejor algoritmo para un problema determinado.